Constraint Satisfaction I

Lecture 8

CS 4880

Kirk Duran

Today

- Why many problems are better described by *requirements* than by action sequences.
- How a constraint satisfaction problem is defined using variables, domains, and constraints.
- How unary, binary, higher-order, and global constraints capture problem structure.
- How partial assignments and consistency define a specialized search space.
- How recursive backtracking systematically constructs a satisfying assignment.
- How map coloring and Sudoku become CSPs under the same general formulation.

Key idea

Constraint satisfaction changes the search abstraction: instead of asking which path reaches a goal, we ask which assignment makes all required relationships true.

Part 1: From General Search to Constraint Satisfaction

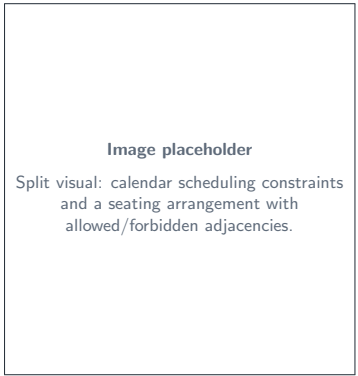
Humans Reason With Constraints Constantly

Many everyday planning tasks are naturally expressed as restrictions on a final configuration.

- Schedule meetings so that required participants do not overlap.
- Seat guests so that incompatible people are separated.
- Assign classrooms while respecting capacity and availability.
- Color neighboring regions differently.

Common structure

We know what relationships must hold, but we may not know the sequence of choices that will produce a valid configuration.



Key idea

Constraint problems are defined more naturally by what must be true than by how the solution is reached.

Map Coloring as a Constraint Problem

Suppose every region of a map must be assigned one of three colors.

- Each region receives exactly one color.
- Adjacent regions may not receive the same color.
- We care about the final legal coloring, not the order in which regions were colored.

Question

Can Australia be colored using only {Red, Green, Blue} while respecting every adjacency?

Scaling Exposes the Real Problem

Small instances may be solvable by inspection. Larger instances expose the combinatorial structure.

- With n variables and d possible values per variable, there are up to d^n complete assignments.
- Most assignments may violate constraints.
- Treating every assignment as an opaque state ignores useful relationships among variables.

Example

Forty-eight states with three candidate colors already permit

$$3^{48}$$

possible complete colorings before constraints are used.

Key idea

The central challenge is not generating assignments; it is exploiting constraints to avoid generating assignments that cannot possibly work.

Why General Search Is an Awkward Fit

Classical search

- Must encode partial assignments as states.
- Generates many illegal or redundant branches.
- Different assignment orders can represent the same partial solution.
- Generic successor generation does not automatically exploit constraint structure.

Local search

- Can begin from a complete but inconsistent assignment.
- May become trapped or fail to establish that no solution exists.
- Requires an objective that quantifies violation quality.
- Does not naturally provide a complete systematic solver.

Key idea

CSP algorithms specialize search around the internal structure of assignments and constraints rather than treating states as indivisible objects.

The Declarative Perspective

A constraint model specifies the conditions that any acceptable solution must satisfy.

- The model describes *what* a solution looks like.
- The solver determines *how* to construct one.
- The same solving machinery can apply to maps, schedules, puzzles, layouts, and allocation problems.

Shift in representation

Search is no longer primarily over paths. It is over assignments to named variables with explicit domains and relationships.

Key idea

Specify what must be true, then let the search procedure determine a consistent assignment.

Part 2: The CSP Model — Variables, Domains, and Constraints

Constraint Satisfaction Problem

Definition

A **constraint satisfaction problem (CSP)** is a triple

$$(X, D, C),$$

where X is a set of variables, D assigns a domain of possible values to each variable, and C is a set of constraints over those variables.

Variables

$$X = \{X_1, \ldots, X_n\}$$

Quantities whose values must be chosen.

Domains

$$D_i = \mathrm{Dom}(X_i)$$

Values currently allowed for X_i.

Constraints

Relations that specify which combinations of values are permitted.

Assignments: The Vocabulary of CSP Search

Partial assignment

Assigns values to only some variables.

$$\{WA = R,\ NT = G\}$$

Complete assignment

Assigns a value to every variable in X.

Consistent assignment

Violates no constraint whose variables have all been assigned.

Solution

A complete assignment that satisfies every constraint in C.

Key idea

Backtracking search grows a partial consistent assignment until it either becomes a complete solution or reaches a contradiction.

Australia as a CSP

Variables

$$X = \{WA, NT, SA, Q, NSW, V, T\}.$$

Domains

For every region,

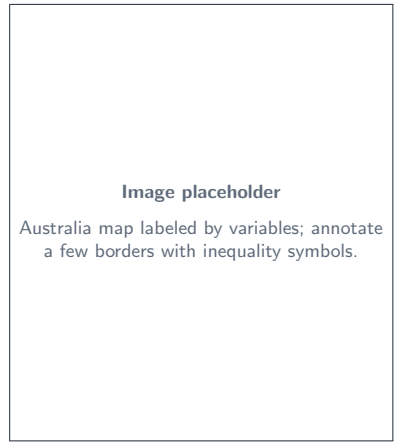
$$D_i = \{R, G, B\}.$$

Binary constraints

Examples include

$$WA \neq NT, \quad WA \neq SA, \quad NT \neq SA,$$

$$NT \neq Q, \quad SA \neq Q, \quad Q \neq NSW.$$



Constraint Graphs

Definition

For a binary CSP, a **constraint graph** has one vertex for each variable and an edge between two variables when a binary constraint relates them.

- Vertices expose the variables that remain to be assigned.
- Edges expose direct dependencies among assignments.
- Graph structure will later guide variable-ordering and inference algorithms.

Australia

Tasmania is isolated in the standard adjacency model, while South Australia is highly connected.

Key idea

The graph tells the solver which decisions can directly constrain one another.

A Solution Is Not a Path

One possible solution is a set of variable-value bindings such as

$$WA = R, \quad NT = G, \quad SA = B,$$

$$Q = R, \quad NSW = G, \quad V = R, \quad T = R.$$

What matters is that:

- every variable has a value from its domain;
- every constraint is satisfied.

Key idea

Two algorithms that reach the same complete satisfying assignment have solved the same CSP even if their internal search histories differ.

Part 3: Constraints Have Structure

Unary and Binary Constraints

Unary constraint

Restricts one variable.

$$X \neq Red$$

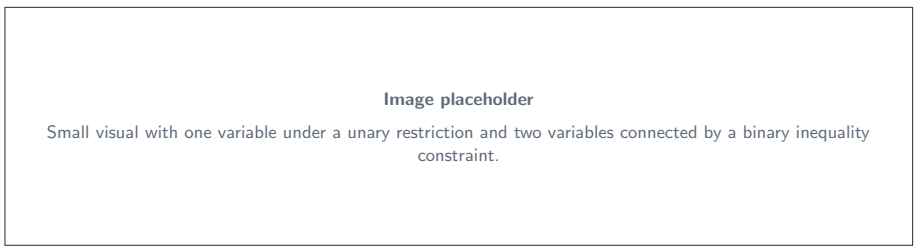
It can often be enforced simply by removing values from the variable's domain.

Binary constraint

Restricts a pair of variables.

$$X \neq Y$$

Binary constraints are naturally represented as edges in a constraint graph.



Higher-Order and Global Constraints

Higher-order constraint

Relates three or more variables simultaneously.
Example: three assigned courses cannot occupy the same room-time combination.

Global constraint

A single high-level relation over a collection of variables.

Example: `AllDifferent`

$$\text{AllDifferent}(X_1, \ldots, X_k)$$

requires every variable in the scope to take a distinct value.

This is a natural representation for rows, columns, and boxes in Sudoku.

Key idea

Constraints are not limited to pairwise inequality; expressive global relations can summarize substantial problem structure.

Constraints Beyond “Not Equal”

Precedence

One activity must occur before another.

$$start(A) + dur(A) \leq start(B)$$

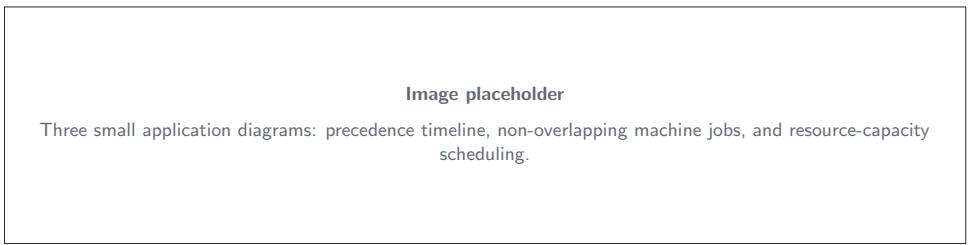
Disjunctive

At least one alternative relationship must hold.

$$A \prec B \ \vee \ B \prec A$$

Resource / capacity

Multiple decisions share a limited resource such as rooms, machines, or staff.



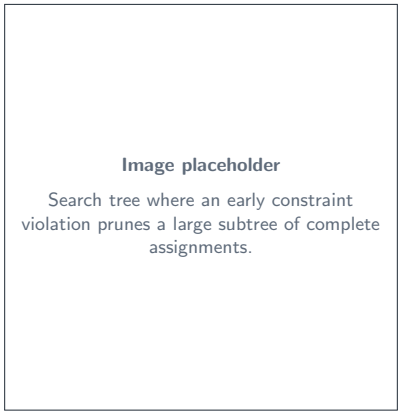
Why Structure Helps Search

In generic search, a state is often treated as an indivisible object.
In a CSP, the solver can inspect the internal structure:

- which variables are already assigned;
- which values remain legal;
- which variables constrain one another;
- which assignments immediately violate a relation.

Consequence

Search can reject an entire family of complete assignments after detecting a contradiction in only a small partial assignment.



Part 4: Consistency — Detecting Impossible Values Early

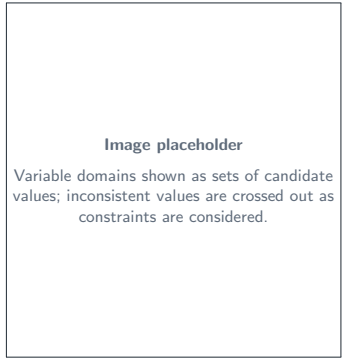
Local Consistency: The Central Question

Constraints let us ask whether values remain compatible with what is currently known.

Reasoning question

Can this candidate value still participate in some assignment that satisfies the relevant constraints?

- If not, the value can be removed from consideration.
- If a variable has no values left, the current search branch is impossible.
- Stronger forms of consistency inspect larger neighborhoods of the constraint graph.



Key idea

Constraint reasoning converts logical incompatibilities into domain reduction before complete assignments are generated.

Node Consistency

Definition

A variable is **node-consistent** when every value remaining in its domain satisfies all unary constraints on that variable.

Suppose

$$D_X = \{1, 2, 3, 4\}$$

with unary constraint

$$X > 2.$$

Enforcing node consistency removes values 1 and 2:

$$D_X \leftarrow \{3, 4\}.$$

Image placeholder

Domain 1,2,3,4 with 1 and 2 crossed out under the unary constraint X ¿ 2.

Arc Consistency — Intuition

Definition

For a directed arc $X_i \to X_j$, the arc is **consistent** when every value in D_i has at least one supporting value in D_j that satisfies the binary constraint between the variables.

- A value without any compatible neighbor value cannot appear in a solution consistent with that arc.
- Removing one value may cause other values elsewhere to lose their support.
- This creates the possibility of *constraint propagation*.

Key idea

Today we use arc consistency only as a reasoning concept; Lecture 9 turns it into an explicit propagation algorithm.

Arc Consistency Example

Let

$$D_X = \{1, 2, 3\}, \qquad D_Y = \{2, 3\}$$

with constraint

$$X < Y.$$

For the arc $X \to Y$:

- $X = 1$ has support: $Y = 2$ or 3.
- $X = 2$ has support: $Y = 3$.
- $X = 3$ has no supporting value.

Therefore

$$D_X \leftarrow \{1, 2\}.$$

Image placeholder

Bipartite support diagram between X=1,2,3 and Y=2,3; show valid X¡Y support edges and cross out X=3.

Part 5: Backtracking Search — A Complete Baseline Solver

Backtracking Search

Definition

Backtracking search constructs a CSP solution incrementally by assigning one unassigned variable at a time and undoing decisions when the current partial assignment cannot be extended consistently.

- Search state: a partial assignment.
- Action: choose an unassigned variable and assign one candidate value.
- Rejection rule: do not continue with an assignment that violates a constraint.
- Goal: a complete consistent assignment.

Key idea

Backtracking is depth-first search specialized to the structure of variable assignments.

Search Over Partial Assignments

The root is the empty assignment:

$$\{\}.$$

A path may grow as

$$\{WA = R\}$$

$$\{WA = R, NT = G\}$$

$$\{WA = R, NT = G, SA = B\}.$$

Any extension that violates a constraint is immediately abandoned.

Image placeholder

Small backtracking tree for Australia: root , then WA assignments, then NT, with inconsistent branches crossed out.

Commutativity Removes Redundant Action Orders

In ordinary path search, different action orders may describe genuinely different paths.
In a CSP,

$$WA = R \text{ then } NT = G$$

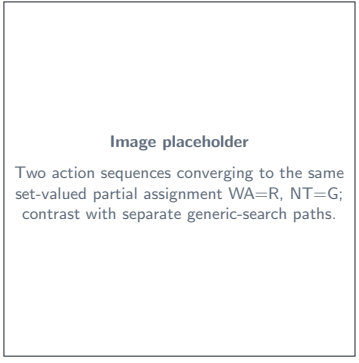
and

$$NT = G \text{ then } WA = R$$

produce the same partial assignment.

Specialization

A backtracking solver chooses exactly one variable at each depth, so it need not explore all permutations of assignment order as separate paths.



Key idea

CSP assignment actions are commutative, allowing a much smaller specialized search tree than a naive action-sequence formulation.

Basic Backtracking Pseudocode

Algorithm

```
BACKTRACK(assignment):
  if assignment is complete:
    return assignment
  X ← choose an unassigned variable
  for value in DOMAIN(X):
    if value is consistent with assignment:
      add X =value to assignment
      result ← BACKTRACK(assignment)
      if result ≠ failure:  return result
      remove X =value from assignment
  return failure
```

Key idea

This is the correct baseline algorithm. Lecture 9 will change how variables and values are selected and add inference between recursive calls.

Worked Example: Beginning the Australia Search

Assume the fixed variable order

$$WA, NT, SA, Q, NSW, V, T$$

and value order

$$R, G, B.$$

Start:

$$\{\}$$

Try

$$WA = R.$$

Next try

$$NT = R.$$

But $WA \neq NT$, so this branch is rejected immediately.

Worked Example: Continuing After Failure

After rejecting $NT = R$, try

$$NT = G.$$

Then consider SA:

- $SA = R$ conflicts with $WA = R$.
- $SA = G$ conflicts with $NT = G$.
- $SA = B$ is consistent with both.

The partial assignment becomes

$$\{WA = R,\ NT = G,\ SA = B\}.$$

Search continues recursively from this smaller feasible region.

Key idea

A single failed consistency test can discard every complete assignment that extends that inconsistent partial assignment.

Completeness Does Not Mean Efficiency

With finite domains, basic backtracking is complete: if a solution exists, exhaustive search will eventually find one. But the worst-case search remains exponential.

$$O(d^n)$$

for n variables with at most d candidate values each.

- Arbitrary variable order can postpone obvious failures.
- Arbitrary value order can destroy future flexibility.
- Basic consistency checks may detect contradictions much later than necessary.

Image placeholder

Large backtracking tree with many explored branches; highlight that correctness does not prevent exponential growth.

Key idea

Backtracking gives us a complete foundation; the next problem is to make that foundation intelligent enough to scale.

Part 6: Modeling a Larger CSP — Sudoku

Sudoku as a Constraint Satisfaction Problem

A standard 9×9 Sudoku can be modeled with one variable per cell.

Variables

$$X_{r,c}, \qquad 1 \le r, c \le 9.$$

Domains

$$D_{r,c} \subseteq \{1, 2, \ldots, 9\}.$$

A given clue fixes the corresponding variable to a singleton domain.

Image placeholder

Partially filled 9x9 Sudoku grid; annotate one empty cell as variable $X_{r,c}$ with domain $\{1, \ldots, 9\}$.

Sudoku Constraints

Every row, column, and 3×3 box must contain distinct digits.

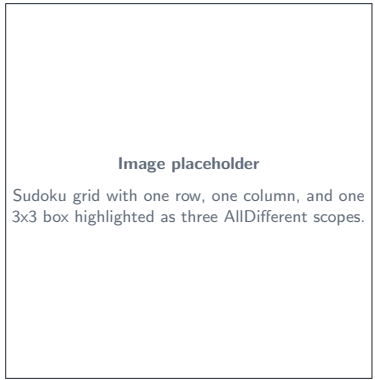
Rows

$$\text{AllDifferent}(X_{r,1}, \ldots, X_{r,9})$$

Columns

$$\text{AllDifferent}(X_{1,c}, \ldots, X_{9,c})$$

The same global constraint is applied to each of the nine boxes.



Key idea

Map coloring and Sudoku look very different visually, but both reduce to assignments constrained by explicit relations.

Naive Backtracking on Sudoku

A basic solver can proceed exactly as before:

1. choose an unassigned cell;
2. try one value from its domain;
3. reject values that violate a row, column, or box constraint;
4. recurse;
5. undo the assignment after failure.

The algorithm is general

Nothing fundamental about backtracking changes between map coloring and Sudoku; only the CSP model changes.

Image placeholder

Sudoku search sequence: choose cell, try digit, mark conflict, undo, then try another digit.

The Problem With Naive Backtracking

Suppose three unassigned variables currently have domains

$$D_A = \{2, 4, 7\},$$

$$D_B = \{5\},$$

$$D_C = \{1, 3, 8, 9\}.$$

A fixed-order solver may choose A or C before B even though B is almost completely determined.

Two questions for Lecture 9

1. Which variable should we assign next?
2. What consequences can we infer immediately after an assignment?

Key idea

The baseline solver treats many decisions as arbitrary even when the current domains and constraint graph contain strong information about what should happen next.

From Basic Search to Intelligent CSP Solving

Today: Foundation	**Next: Efficient CSP Solving**
Variables, domains, constraints	Minimum Remaining Values (MRV)
Constraint graphs	Degree heuristic
Partial and complete assignments	Least Constraining Value (LCV)
Consistency checking	Forward checking
Basic backtracking	Arc consistency and propagation
Map coloring and Sudoku models	Maintaining Arc Consistency (MAC)

Key idea

Lecture 8 establishes a complete solver; Lecture 9 improves its search order and adds inference so that contradictions are discovered much earlier.

Takeaways

- A CSP is defined by variables X, domains D, and constraints C.
- A solution is a complete assignment that satisfies every constraint.
- Constraint graphs expose dependencies among variables in binary CSPs.
- Constraint types include unary, binary, higher-order, and global relations such as `AllDifferent`.
- Local consistency asks whether candidate values remain supportable under the known constraints.
- Backtracking performs depth-first search over partial assignments and is complete for finite CSPs.
- Basic backtracking can still require exponential search because it makes uninformed ordering decisions and performs only limited inference.

Key idea

The power of the CSP representation is that search can reason about the internal structure of a partial solution; the next lecture turns that structure into aggressive pruning and propagation.